

UNIVERSIDAD EUROPEA DE MADRID

ESCUELA DE ARQUITECTURA, INGENIERÍA Y DISEÑO

**MÁSTER EN FORMACIÓN PERMANENTE EN INTELIGENCIA ARTIFICIAL
APLICADA**

TRABAJO FIN DE MÁSTER

**Quirón: un editor nativo con índice de código
verificable y una red neuronal local para el aporte de
contexto a modelos de lenguaje**

Lorenzo Juan Santacreu Pascual

Dirigido por

[COMPLETAR — Nombre del director/a]

CURSO 2025–2026

TÍTULO: Quirón: un editor nativo con índice de código verificable y una red neuronal local para el aporte de contexto a modelos de lenguaje

AUTOR: Lorenzo Juan Santacreu Pascual

TITULACIÓN: Máster en Formación Permanente en Inteligencia Artificial Aplicada

DIRECTOR/ES DEL PROYECTO: [COMPLETAR — Nombre del director/a]

FECHA: Septiembre de 2026

Índice

RESUMEN	7
ABSTRACT	8
TABLA RESUMEN	9
Capítulo 1. RESUMEN DEL PROYECTO	10
1.1 Contexto y justificación	10
1.2 Planteamiento del problema	10
1.3 Objetivos del proyecto	10
1.4 Resultados obtenidos	10
1.5 Estructura de la memoria	10
Capítulo 2. ANTECEDENTES / ESTADO DEL ARTE	11
2.1 Estado del arte	11
2.1.1 Recuperación de contexto en repositorios	11
2.1.2 Representación, generación y almacenes	11
2.1.3 Alternativas para una red propia	12
2.2 Contexto y justificación	12
2.3 Planteamiento del problema	13
Capítulo 3. OBJETIVOS	14
3.1 Objetivo general	14
3.2 Objetivos específicos	14
3.3 Beneficios del proyecto	15
Capítulo 4. DESARROLLO DEL PROYECTO	16
4.1 Planificación	16
4.2 Solución, metodología y herramientas	17
4.2.1 Memoria de eventos e índice de código	17
4.2.2 Identidad y recorrido del proyecto	18
4.2.3 Archivos, lógicas y fichas	18
4.2.4 Grafo y recuperación	19

4.2.5 Editor y herramientas	20
4.2.6 Proveedores y suscripciones	21
4.2.7 Worker descargable y adaptación al dispositivo	22
4.2.8 Selección de modelos preentrenados y delimitación del entrenamiento propio	22
4.2.9 Metodología y reproducibilidad	23
4.3 Recursos	23
4.4 Presupuesto	24
4.5 Viabilidad y despliegue	24
4.6 Resultados y evaluación	25
4.6.1 Pruebas automatizadas	25
4.6.2 Integración del worker y del editor	26
4.6.3 Evaluación exploratoria de localización	26
4.6.4 Fidelidad de las fichas	28
4.6.5 Proveedores desde la interfaz	28
4.6.6 Validación pendiente	29
Capítulo 5. DISCUSIÓN	30
Capítulo 6. CONCLUSIONES	31
6.1 Conclusiones del trabajo	31
6.2 Conclusiones personales	31
Capítulo 7. FUTURAS LÍNEAS DE TRABAJO	32
Capítulo 8. REFERENCIAS	33
Capítulo 9. ANEXOS Y TRAZABILIDAD	36
Anexo A. Estudio de alternativas para la red obrera	37
Estudio: la matemática de la red obrera de Quirón	37
Cómo leer las etiquetas de evidencia	37
0. Marco del diseño inicial	38
FAMILIA 1 — Atención eficiente y gestión del KV-cache	38
1.1 Multi-head Latent Attention (MLA) — comprimir el KV · Verificado	38
1.2 Native Sparse Attention (NSA) — atención dispersa entrenable · Verificado	38
1.3 Atención lineal / lightning + SSE — estado de tamaño fijo · Verificado	39

FAMILIA 2 — Mezcla de expertos (MoE) y enrutado	40
2.1 DeepSeekMoE: experto compartido + expertos de grano fino · Verificado	40
2.2 Balanceo de carga SIN pérdida auxiliar · Verificado	40
FAMILIA 3 — Destilación profesor → estudiante	41
3.1 KD clásica (Hinton): soft targets con temperatura · Fundamental	41
3.2 Sequence-Level KD — la vía viable sin logits · Verificado	41
3.3 Reverse-KL (MiniLLM) y GKD — por qué NO nos sirven · Verificado	42
3.4 Matiz: ¿reverse-KL es realmente mejor? · Cuestión abierta	42
FAMILIA 4 — Cuantización y representación	42
4.1 AWQ — cuantizar protegiendo canales por activación · Verificado	42
4.2 GPTQ — cuantización PTQ de segundo orden · Verificado	43
4.3 InfoNCE — la pérdida que da buenos embeddings · Fundamental	43
4.4 Last-token pooling — de decoder causal a embedding · Fundamental	43
4.5 Matryoshka (MRL) — un vector, muchas dimensiones · Fundamental	43
Síntesis — la red obrera propuesta	44
Qué es reutilizable y qué es solo principio de diseño	45
Cuestiones abiertas	45
Referencias del anexo	46
Síntesis de la propuesta inicial	46

Índice de figuras

Figura 1. Cronograma del proyecto: actividades realizadas entre febrero y septiembre de 2026	16
Figura 2. Componentes de Quirón y flujo de datos entre editor, cerebro, almacenes, vectorizador y asistentes	17
Figura 3. Pantalla de vectorización al abrir un proyecto: el editor muestra el progreso del worker y queda vigilando cambios	19
Figura 4. Pantalla de bienvenida del editor	20
Figura 5. Ronda de chat con Claude sobre un proyecto sintético: el asistente lee el archivo mediante la herramienta del editor y la respuesta enlaza las fuentes con ruta y líneas	21
Figura 6. Selector de modelos de la paleta Agentes con los alias de Claude y el catálogo de la sesión de Codex	21

Índice de tablas

Tabla 1. Datos generales del trabajo	9
Tabla 2. Aspectos cubiertos por Quirón frente a los antecedentes	11
Tabla 3. Objetivos específicos, planteamiento inicial y estado al cierre	14
Tabla 4. Etapas del proyecto	16
Tabla 5. Recursos empleados	23
Tabla 6. Presupuesto del proyecto	24
Tabla 7. Pruebas automatizadas de la revisión entregada	25
Tabla 8. Evidencias funcionales registradas	26
Tabla 9. Resultados de la evaluación de localización sobre diez preguntas	27
Tabla 10. Qué es reutilizable y qué es solo principio de diseño	45

RESUMEN

Los asistentes de programación necesitan localizar información relevante dentro de un repositorio para responder y proponer cambios. Este trabajo presenta Quirón, un editor nativo escrito en Rust que integra un índice de código mantenido por un worker local. Al abrir un proyecto, el sistema detecta sus archivos, extrae unidades sintácticas de Rust, genera fichas descriptivas y produce representaciones vectoriales para su consulta. La configuración inicial utiliza Qwen2.5-Coder-1.5B cuantizado para redactar las fichas y BGE-M3 para vectorizarlas; ambos modelos se ejecutan en el equipo del usuario. Qdrant almacena los vectores y Neo4j representa la pertenencia de las unidades y relaciones aproximadas de llamadas. Cada ficha conserva proyecto, ruta, símbolo, líneas y hash del archivo, y la recuperación comprueba su vigencia frente al código actual antes de entregarla al asistente.

La aportación consiste en integrar ese recorrido con el editor, con la descarga y selección de modelos compatibles y con distintas conexiones a asistentes (Claude, Codex, servidores compatibles con OpenAI y Ollama). El entrenamiento de una red propia, previsto en el planteamiento inicial, quedó fuera del alcance final por razones de recursos, plazo y prioridad de validación, y se documenta como estudio de alternativas. Las evidencias registradas cubren indexación incremental, aislamiento entre proyectos, cambios, borrados, uso de herramientas con proveedores reales y simulados y una evaluación exploratoria de localización; también muestran limitaciones de fidelidad en los resúmenes generados. La evaluación acredita un prototipo funcional; no permite afirmar todavía ahorro de tokens en tareas completas, prevención de duplicados ni análisis exhaustivo de dependencias.

Palabras clave: modelos de lenguaje, recuperación aumentada, índice de código, inferencia local, editor nativo, trazabilidad.

ABSTRACT

Programming assistants need to locate relevant information within a repository to answer questions and propose changes. This work presents Quirón, a native Rust editor that integrates a code index maintained by a local worker. When a project is opened, the system discovers its files, extracts Rust syntax units, generates descriptive records and produces vector representations for retrieval. The initial configuration uses quantized Qwen2.5-Coder-1.5B for descriptions and BGE-M3 for embeddings; both run on the user's machine. Qdrant stores the vectors, while Neo4j represents unit membership and approximate call relationships. Each record keeps the project, path, symbol, line range and file hash, and retrieval checks its freshness against the current source before handing it to the assistant.

The contribution is the integration of this workflow with the editor, with downloadable and replaceable compatible models, and with several assistant connections (Claude, Codex, OpenAI-compatible servers and Ollama). Training a proprietary worker model, part of the initial plan, was excluded from the final scope because of resource constraints, the available schedule and validation priorities, and is documented as a study of alternatives. Recorded evidence covers incremental indexing, isolation between projects, updates, deletions and tool interactions through real and simulated providers, together with an exploratory retrieval evaluation. It also exposes limitations in the fidelity of generated descriptions. The available evaluation supports a functional prototype; it does not yet establish token savings for complete tasks, duplicate prevention or exhaustive dependency analysis.

Keywords: language models, retrieval augmentation, code indexing, local inference, native editor, traceability.

TABLA RESUMEN

Campo	Datos
Nombre y apellidos	Lorenzo Juan Santacreu Pascual
Título del proyecto	Quirón: un editor nativo con índice de código verificable y una red neuronal local para el aporte de contexto a modelos de lenguaje
Directores del proyecto	[COMPLETAR]
El proyecto se ha realizado en colaboración de una empresa o a petición de una empresa	NO
El proyecto ha implementado un producto	SÍ: prototipo de editor nativo con índice de código local
El proyecto ha consistido en el desarrollo de una investigación o innovación	NO: ingeniería aplicada; el estudio de alternativas del Anexo A no constituye una investigación experimental
Objetivo general del proyecto	Construir y evaluar un prototipo de editor nativo que mantenga un índice de código mediante un worker local configurable y aporte al asistente referencias comprobables contra los archivos del proyecto

Tabla 1. Datos generales del trabajo

Capítulo 1. RESUMEN DEL PROYECTO

1.1 Contexto y justificación

Trabajar sobre proyectos grandes exige saber qué contiene cada archivo y dónde se implementa cada responsabilidad. La experiencia con asistentes de programación durante el máster mostró tres problemas recurrentes: el asistente reescribe lógica que ya existe porque no la encuentra, interpreta mal el código por falta de contexto y gasta buena parte de su ventana en leer archivos que no necesita. De ahí nace la idea de un mapa persistente del proyecto, mantenido en local, que facilite localizar código y seleccionar contexto.

1.2 Planteamiento del problema

La pregunta de implementación es: **¿puede un editor mantener un mapa consultable de los archivos y las lógicas de un proyecto mediante un worker local, y entregar al asistente referencias cuya vigencia se compruebe contra el código?** La pregunta experimental es si ese mapa mejora la localización y reduce el contexto necesario sin perder calidad. Este trabajo, un desarrollo de producto sin colaboración empresarial, responde a la primera con un prototipo y aporta una evaluación exploratoria de la segunda.

1.3 Objetivos del proyecto

El alcance final comprende el editor con guardia de rutas, la indexación incremental con fichas de un modelo preentrenado y embeddings, las proyecciones en Qdrant y Neo4j, la recuperación conectada al chat y la selección de modelos para adaptar el worker al equipo; el Capítulo 3 detalla los objetivos y su evolución.

1.4 Resultados obtenidos

Abrir una carpeta produce en segundo plano un índice de fichas vectorizadas por proyecto que el chat consulta y devuelve como fuentes clicables. Las pruebas registradas cubren apertura, consultas, cambios, borrados, exclusión de archivos sensibles y aislamiento entre proyectos, además de rondas completas desde la interfaz con Claude y Codex. En una evaluación exploratoria de localización con diez preguntas, el objetivo apareció entre los cinco primeros resultados en todos los casos. Los errores del generador se conservaron y motivaron fichas con citas verificadas por el programa; la revisión final corrigió la integridad del registro de eventos y documentó los límites restantes.

1.5 Estructura de la memoria

Los capítulos 2 a 9 presentan, por este orden, los antecedentes, los objetivos, el desarrollo y la evaluación, la discusión, las conclusiones, el trabajo futuro, las referencias y los anexos con las evidencias.

Capítulo 2. ANTECEDENTES / ESTADO DEL ARTE

2.1 Estado del arte

Los asistentes de programación necesitan recuperar del repositorio el contexto que no cabe en su ventana. La literatura analizada se organiza en tres frentes: la recuperación de contexto en repositorios, los modelos de representación y generación con sus almacenes, y las arquitecturas que se estudiaron para una red propia.

2.1.1 Recuperación de contexto en repositorios

Existen trabajos previos que recuperan información del repositorio para asistir a modelos de código. RepoCoder combina recuperación por similitud y generación iterativa [25]. GraphCoder utiliza grafos de contexto de código para recuperar fragmentos [26]. RepoGraph ofrece una estructura de repositorio que sirve de apoyo a agentes de ingeniería de software [27]. Estos antecedentes sitúan a Quirón: la recuperación de código mediante grafos y vectores no es una invención de este trabajo.

Lo que se estudia aquí es la integración de un worker local configurable con un editor nativo, proyecciones incrementales y comprobación de vigencia de las fichas. No se presenta una comparación experimental que establezca superioridad frente a los sistemas anteriores. La matriz siguiente describe el alcance de Quirón, no una clasificación exhaustiva de productos.

Aspecto	Aportación desarrollada en Quirón
Localización	Consulta de fichas con ruta, símbolo, líneas y hash
Mantenimiento	Monitor incremental del proyecto y retirada de unidades desaparecidas
Modelos	Generador local sustituible y embeddings independientes
Interfaz	Editor nativo con estado del worker, proveedores y fuentes recuperadas
Dependencias	Pertenencia y aproximación de llamadas Rust; cobertura incompleta
Evaluación	Integración y casos exploratorios; falta comparación de tareas completas

Tabla 2. Aspectos cubiertos por Quirón frente a los antecedentes

2.1.2 Representación, generación y almacenes

Un generador de texto y un modelo de embeddings cumplen tareas distintas. BGE-M3 [19][29] proporciona las representaciones utilizadas por esta implementación. Qwen2.5-Coder [23] genera descripciones breves. La calidad de estas descripciones puede influir en la recuperación, pero el tamaño del generador no garantiza una mejora en todas las tareas.

La literatura sobre InfoNCE [13], last-token pooling [14][15] y Matryoshka [16] sirvió de base para explorar modelos de representación propios. El uso del estado del último token no

acredita por sí solo calidad de embeddings, y las representaciones truncables requieren un entrenamiento apropiado: dos modelos con vectores de la misma dimensión no comparten necesariamente un espacio semántico. En esta versión no se entrena una representación propia ni se demuestra que los vectores de BGE-M3 puedan truncarse conservando calidad.

Qdrant [17] almacena los vectores y permite restringir candidatos mediante filtros de carga útil. Neo4j [18] almacena nodos y relaciones consultables. La identidad de las unidades permite vincular ambas proyecciones. La existencia de un grafo no implica que sus relaciones resuelvan todos los tipos, importaciones o llamadas del programa.

2.1.3 Alternativas para una red propia

El estudio inicial revisó MLA [1][2], atención dispersa [3], mecanismos lineales [4–7], mezcla de expertos [8][9], destilación [10–12c] y cuantización [20][21]. Su resultado se conserva en el Anexo A como exploración de alternativas. Ninguno de esos mecanismos se implementó como un nuevo backbone entrenado en este trabajo.

La recurrencia de estado fijo puede tener coste constante por paso respecto a la longitud del contexto, para dimensiones fijas. Esta propiedad no se extiende al modelo completo si se añaden capas de atención completa sobre todo el historial. La exportación y el rendimiento de una arquitectura híbrida en ONNX/Rust quedaron como cuestiones por medir.

Sequence-Level KD [11] permite utilizar respuestas textuales de un profesor. QLoRA [24] muestra que es posible ajustar modelos preentrenados reduciendo el consumo de memoria. La decisión de no entrenar en esta versión es de alcance y viabilidad; no supone que cualquier ajuste resulte imposible con el equipo disponible.

2.2 Contexto y justificación

El proyecto nace de la práctica diaria con asistentes de programación sobre repositorios propios de tamaño medio, y su aplicabilidad prevista es el uso individual de un desarrollador en su equipo: el índice se construye y se consulta en local, y el asistente puede ser cualquiera de los proveedores admitidos. Se busca facilitar la navegación por responsabilidades del código, conservar referencias comprobables y adaptar la inferencia al equipo del usuario. El ahorro económico y la prevención de lógica duplicada permanecen como hipótesis de aplicación. El cómputo local tiene costes de descarga, almacenamiento, memoria, energía y mantenimiento; disponer del hardware no los elimina. Las suscripciones y las conexiones por API son vías de acceso diferentes, con condiciones y límites propios.

2.3 Planteamiento del problema

Los sistemas revisados resuelven la recuperación desde el lado del agente o de la herramienta de completado: construyen el contexto por petición, sobre un repositorio que tratan como estático, y no ofrecen al usuario un mapa persistente que evolucione con el código ni una comprobación de que lo recuperado sigue vigente. Tampoco separan el trabajo continuo de descripción, que puede asumir un modelo pequeño en local, del razonamiento que se pide a un modelo grande. La necesidad detectada es un componente que viva en el editor, mantenga el índice mientras se trabaja, aísla cada proyecto y entregue referencias verificables. De esa necesidad se derivan los objetivos del Capítulo 3.

Capítulo 3. OBJETIVOS

3.1 Objetivo general

El objetivo general del presente trabajo es **construir y evaluar un prototipo de editor nativo que mantenga un índice de código mediante un worker local configurable y aporte al asistente referencias comprobables contra los archivos del proyecto**. La versión inicial del proyecto incluía el entrenamiento de una red propia, un historial completo de cambios y un análisis amplio de dependencias. Durante el desarrollo se priorizó el recorrido funcional con modelos preentrenados, y se conserva la numeración de los objetivos para hacer visible esa evolución en lugar de presentar una reformulación como cumplimiento retroactivo del planteamiento inicial.

3.2 Objetivos específicos

Objetivo	Planteamiento inicial	Alcance y estado al cierre
OE1	Construir un registro inmutable de eventos y dos proyecciones reconstruibles desde él	Registro encadenado v2 con hash de todos los campos, transacciones y anclaje compatible; herramientas de replay de eventos; reconstrucción completa del índice de código pendiente
OE2	Desarrollar un editor nativo confinado al proyecto abierto	Editor y guardia de rutas implementados, con pruebas; quedan límites ante carreras del sistema de archivos
OE3	Implementar un indexador incremental de unidades Archivo, Lógica y Cambio	Archivo y lógica Rust operativos; tipo Cambio definido sin historial completo integrado
OE4	Construir un grafo de dependencias resuelto por análisis estático	Pertenencia y llamadas aproximadas Rust; resolución de tipos, imports y dependencias completas pendientes
OE5	Recuperar contexto por vector, expansión por grafo, reranker y presupuesto de tokens	Filtro en la búsqueda de código y expansión limitada implementados; reranker y presupuesto global pendientes
OE6	Estudiar mecanismos para una red propia exportable a ONNX	Estudio de alternativas documentado (Anexo A); no se construyó un nuevo backbone
OE7	Entrenar por destilación y desplegar una red propia como worker	Entrenamiento excluido; objetivo reformulado a integrar, descargar y seleccionar modelos preentrenados compatibles
OE8	Medir ahorro de contexto, recuperación, aislamiento y fidelidad	Evidencias funcionales y evaluación exploratoria; la hipótesis de ahorro en tareas completas no está demostrada

Tabla 3. Objetivos específicos, planteamiento inicial y estado al cierre

3.3 Beneficios del proyecto

El índice facilita localizar código que podría reutilizarse y revisar la procedencia de la información recuperada. Un hash coincidente acredita la vigencia del archivo, no la corrección de su resumen. Quirón no dispone de un detector automático de lógica duplicada conectado al editor ni de resúmenes consolidados de responsabilidades por carpeta. Esas extensiones no forman parte de los resultados de esta versión.

Capítulo 4. DESARROLLO DEL PROYECTO

4.1 Planificación

La planificación inicial situaba la infraestructura y el editor antes del entrenamiento. En septiembre se sustituyó el entrenamiento propio por la integración de modelos preentrenados y se priorizó el cierre funcional: monitor por proyecto, fichas y vectores, proveedores desde la interfaz, paquete de instalación y evaluación. La tabla resume las etapas.

Etapas	Periodo	Trabajo realizado
Infraestructura	Febrero–junio de 2026	Servicios locales, registro de eventos, editor nativo y guardia de rutas
Memoria de eventos y estudio	Junio–julio de 2026	Memoria de eventos con proyecciones, diagnóstico de la recuperación y estudio matemático de la red obrera (Anexo A)
Índice de código	Agosto–septiembre de 2026	Indexador tree-sitter, worker Qwen/BGE-M3, colecciones por proyecto, grafo de llamadas, recuperación conectada al chat
Interfaz y proveedores	Septiembre de 2026	Rediseño de la interfaz, paleta Agentes, conexiones Claude, Codex, compatibles y Ollama, catálogo del vectorizador
Cierre	10–13 de septiembre de 2026	Contraste entre código y memoria, ledger v2, fichas v5, paquete portátil, evaluación exploratoria, limpieza del código y redacción final

Tabla 4. Etapas del proyecto

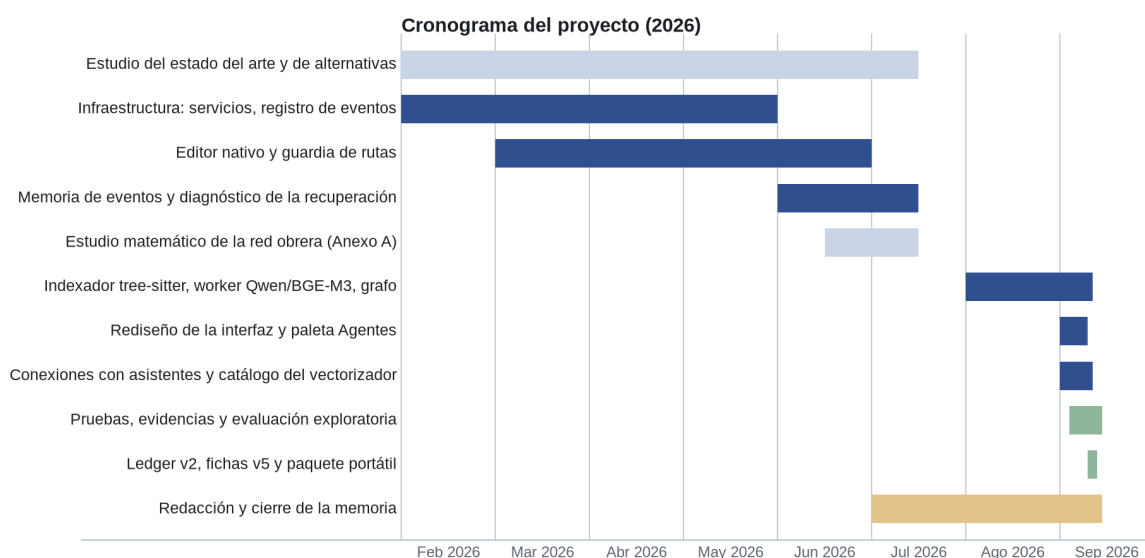


Figura 1. Cronograma del proyecto: actividades realizadas entre febrero y septiembre de 2026

El esfuerzo total en horas se recoge en el presupuesto del apartado 4.4. El historial Git del repositorio de entrega comienza el 29 de agosto de 2026 con la reorganización documental del proyecto; el código anterior a esa fecha se incorporó en ese primer commit.

La entrega se realiza por repositorio, según lo acordado con el tutor: **[COMPLETAR — URL del repositorio de entrega]**. La revisión entregada está identificada con la etiqueta entrega-tfm; el paquete binario incluye un inventario de hashes (BUILD.json y SHA256SUMS).

4.2 Solución, metodología y herramientas

Quirón se compone de cuatro piezas: el editor nativo, el cerebro (índice, registro de eventos y API local), la pasarela hacia los asistentes y el vectorizador local. En esta memoria «worker» designa el proceso del cerebro que mantiene el índice; en la interfaz aparece como «vectorizador», y el modelo Qwen que redacta las fichas es su generador. La figura resume los componentes y los flujos de datos entre ellos.

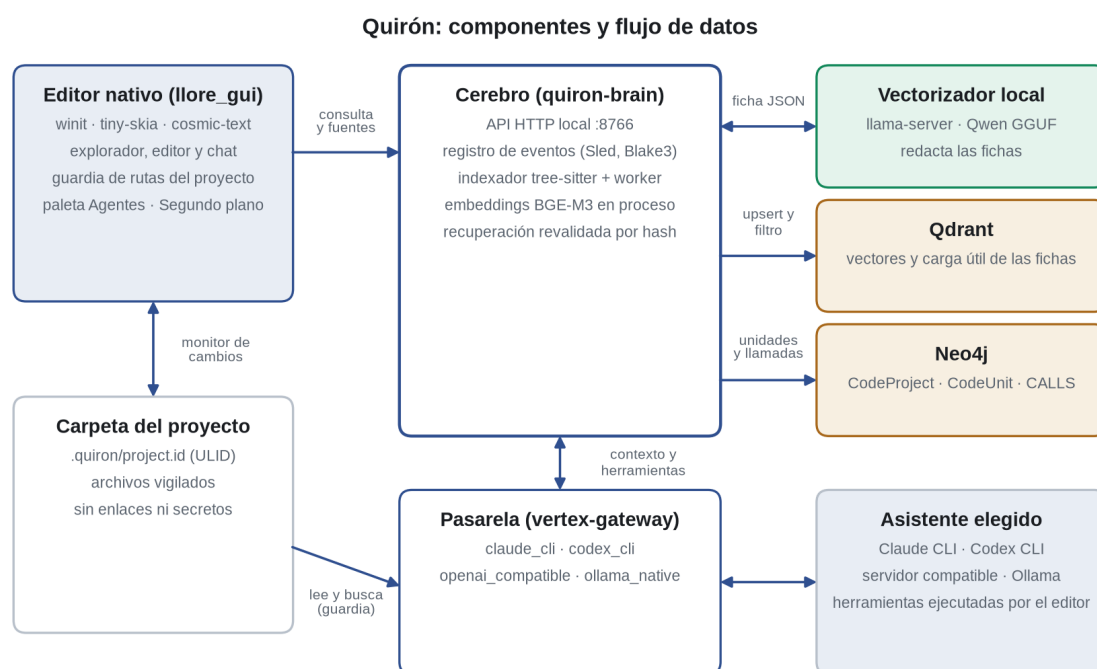


Figura 2. Componentes de Quirón y flujo de datos entre editor, cerebro, almacenes, vectorizador y asistentes

4.2.1 Memoria de eventos e índice de código

El cerebro utiliza Sled [32] como almacenamiento local; el nombre histórico del módulo storage/rocks.rs no significa que se utilice RocksDB. Los eventos se encadenan mediante Blake3. Existen envolturas de memoria con estados y relaciones de sustitución o retractación, y herramientas de reconstrucción de proyecciones de eventos.

El índice de código mantiene su manifiesto y la caché de fichas y vectores en Sled, y obtiene el contenido de los archivos del disco. Confirma un archivo después de recibir el acuse de

Qdrant y Neo4j, y comprueba periódicamente la presencia de las unidades para reparar determinadas pérdidas mediante la caché. Este mecanismo no equivale a reconstruir íntegramente el índice desde el registro de eventos: no se registran eventos suficientes para reproducir todo su historial.

La revisión final del registro encontró que la versión inicial del hash no cubría todos los campos del evento y que los índices se escribían en operaciones separadas. La corrección, denominada v2, calcula el hash sobre la serialización completa del evento (salvo su propio hash), con separación de dominio y longitudes delimitadas. Evento, índices, versión, anclaje y cabecera se escriben en una transacción de Sled que solicita persistencia antes de confirmar. Los escritores comparten un bloqueo y los identificadores duplicados se rechazan, también dentro de un lote. La verificación propaga los registros ilegibles en lugar de omitirlos.

Los eventos históricos conservan sus bytes y su algoritmo. El primer evento v2 incorpora un anclaje del contenido completo observado al migrar, lo que permite detectar cambios posteriores; no demuestra la originalidad del contenido anterior a ese momento. La prueba de interrupción termina un proceso después de confirmar lotes y verifica su recuperación íntegra; no se ha simulado una pérdida de alimentación. Tampoco se acredita resistencia frente a un actor capaz de reescribir toda la base y su cabecera, al no existir una raíz de confianza externa. Las envolturas derivadas de memoria siguen fuera de la transacción autoritativa.

4.2.2 Identidad y recorrido del proyecto

La identidad de cada proyecto es un ULID guardado en `.quiron/project.id`; la aplicación migra el estado histórico de `.llore`. El worker exige una raíz válida y una identidad coincidente. Una copia con la misma identidad no puede monitorizarse simultáneamente como otro proyecto independiente. La separación entre proyectos se verifica con identidades distintas.

El recorrido excluye enlaces simbólicos, nombres sensibles y directorios generados. Cuando la raíz es un repositorio Git, respeta el conjunto de archivos seguidos o no ignorados por Git. Sus límites son 512 KiB por archivo, 10 000 archivos y 64 MiB de texto por barrido. Las reglas se basan en rutas y nombres; no garantizan detectar credenciales incrustadas dentro de cualquier archivo de código.

4.2.3 Archivos, lógicas y fichas

Se genera una unidad de archivo para las extensiones admitidas. Tree-sitter [31] añade unidades de lógica Rust: funciones, métodos, estructuras, enumeraciones y traits, con firma y líneas. La cobertura no incluye toda construcción del lenguaje, y los demás lenguajes tienen por ahora ficha de archivo.

El worker proporciona al generador hasta 6 000 caracteres de la unidad, numerados por línea, y las definiciones de constantes Rust referenciadas cuando caben en un contexto

acotado. Esas definiciones participan en la huella de caché, de modo que un cambio en ellas invalida la ficha. Las entradas recortadas se marcan como parciales. La respuesta solicitada es un propósito en una frase completa, el contexto no resuelto y entre una y tres líneas de evidencia; el programa copia los fragmentos citados desde el código y convierte sus líneas a posiciones del archivo, de modo que el modelo no redacta las citas.

Se rechazan frases incompletas, referencias inexistentes, cifras ausentes del contexto y determinadas menciones de lenguajes sin respaldo en la entrada, además de salidas inválidas o que repiten instrucciones. En esos casos se guarda una ficha estructural con el motivo y `summary_origin=parser`; las aceptadas llevan `model`. Los errores de conexión son reintentables. Las reglas y las citas verifican restricciones concretas, no toda afirmación semántica de la descripción.

BGE-M3 [19][29], ejecutado en proceso sobre ONNX Runtime [22], vectoriza ruta, símbolo, firma y descripción. La ficha mantiene la relación con el archivo sin copiar todo su código al texto del vector. El extractor calcula además una huella del cuerpo normalizado por espacios, pero no existe un comparador automático de duplicados conectado a esa huella.



Figura 3. Pantalla de vectorización al abrir un proyecto: el editor muestra el progreso del worker y queda vigilando cambios

4.2.4 Grafo y recuperación

Neo4j contiene los nodos `CodeProject` y `CodeUnit` y las relaciones `HAS_UNIT`, `DEFINED_IN` y `CALLS`. Las llamadas se extraen sintácticamente y se enlazan mediante reglas por nombre y archivo. Se omiten los casos ambiguos según esas reglas, pero una coincidencia única de nombre no prueba la resolución semántica del destino: se pierden determinados prefijos de módulo y no se resuelve el tipo del receptor. No se ofrecen garantías de análisis de impacto completo.

La búsqueda de código aplica dentro de Qdrant filtros de proyecto y de versiones de modelos, y revalida ruta y hash antes de devolver cada acierto. Añade un vecino por llamadas cuando puede obtenerlo del grafo. La consulta de vecinos filtra explícitamente proyecto y versiones de modelos en la semilla y en el vecino, y conserva la ficha JSON y su indicador de entrada parcial; estas condiciones se comprobaron introduciendo aristas entre proyectos de contenido idéntico y nodos de versiones anteriores. No se ejecuta un reranker de fichas ni se aplica un presupuesto global medido con el tokenizador del asistente. La memoria de eventos utiliza otro recorrido, en el que parte del filtrado por proyecto sigue ocurriendo después de buscar; las garantías verificadas del índice de código no se extienden automáticamente a todos los endpoints.

4.2.5 Editor y herramientas

El editor utiliza `winit`, `tiny-skia`, `softbuffer`, `taffy` y `cosmic-text`, con las tipografías incluidas en el producto. Permite abrir proyectos, editar archivos y consultar el chat; muestra el estado del worker en el panel «Segundo plano» y las fuentes recibidas como enlaces al archivo y a la línea. La paleta «Agentes» integra la selección de proveedores y del modelo del worker, y el manual de uso se abre desde la propia aplicación.

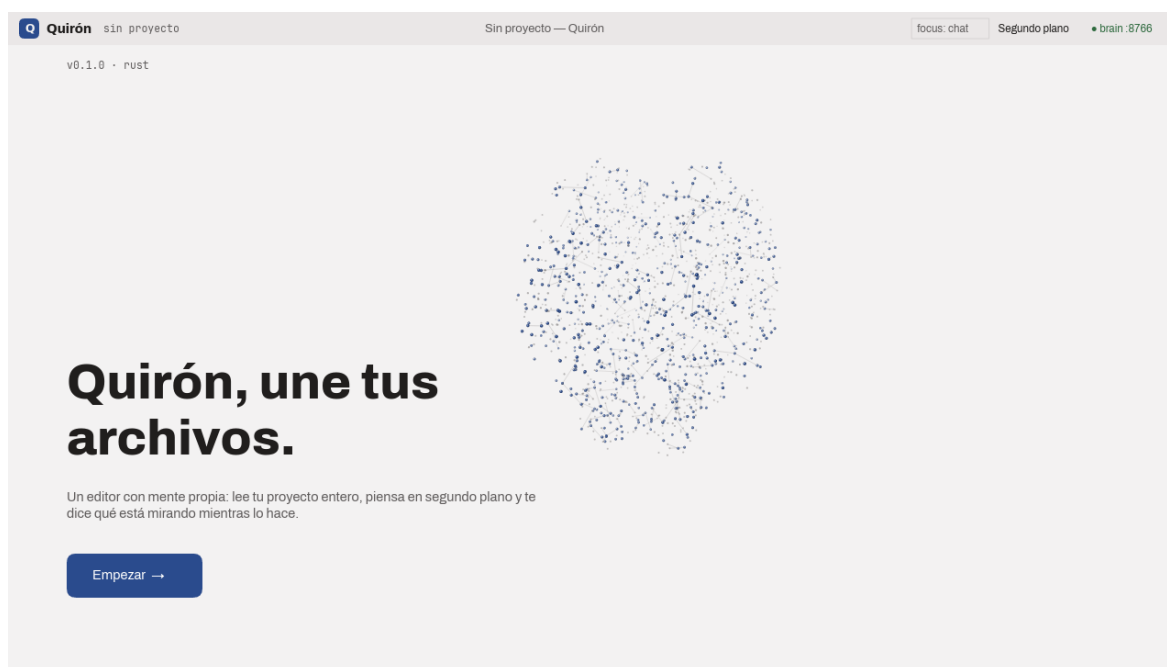


Figura 4. Pantalla de bienvenida del editor

Las herramientas ofrecidas al asistente son lectura de archivos, listado y búsqueda textual. Se ejecutan mediante la guardia del proyecto y con límites de tamaño de salida. Su existencia no implica edición autónoma ni que cada respuesta consulte todos los archivos pertinentes. Persisten carreras posibles entre inspeccionar una ruta y abrirla; el modelo de uso evaluado es un escritorio local de un solo usuario.

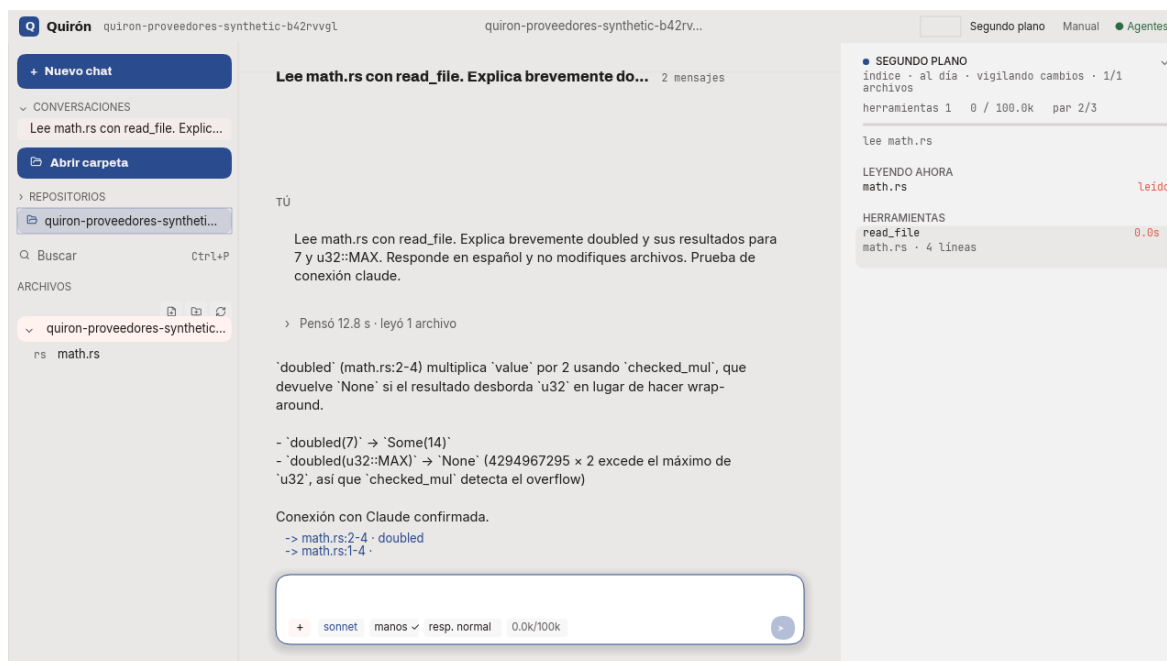


Figura 5. Ronda de chat con Claude sobre un proyecto sintético: el asistente lee el archivo mediante la herramienta del editor y la respuesta enlaza las fuentes con ruta y líneas

4.2.6 Proveedores y suscripciones

La pasarela admite cuatro conexiones: la CLI oficial de Claude (`claude_cli`), la CLI oficial de Codex (`codex_cli`), servidores compatibles con la API de OpenAI (`openai_compatible`, que cubre OpenAI, un servidor en la red local u Ollama por su interfaz compatible) y la API nativa de Ollama (`ollama_native`). Una suscripción no proporciona un identificador universal de acceso: cada adaptador tiene su propio mecanismo y restricciones, y las conexiones compatibles pueden requerir una clave de API. El selector reúne los alias de Claude (Sonnet, Opus y Haiku) y el catálogo de modelos visibles en la sesión de Codex, que puede actualizarse desde la interfaz. Cada conversación transmite su proveedor y modelo sin cambiar la configuración del servicio, de modo que elegir Claude o Codex no detiene el worker. En Codex se transmite también el esfuerzo de razonamiento.



Figura 6. Selector de modelos de la paleta Agentes con los alias de Claude y el catálogo de la sesión de Codex

Codex CLI se ejecuta en un directorio temporal, sin la configuración personal del usuario y con sus herramientas nativas de acceso a archivos, comandos y servicios desactivadas; la respuesta sigue un contrato JSON cuyas llamadas a herramientas ejecuta la guardia del editor. Quirón elimina del entorno del subprocesso las credenciales del cerebro, de los almacenes y de otros proveedores. Es una conversación propia de Quirón, sin heredar el hilo del IDE.

Las evidencias incluyen proveedores reales y un servidor simulado que comprueba el transporte de herramientas. Una prueba con ese servidor verifica integración, no la calidad de un modelo comercial. Los resultados históricos no garantizan disponibilidad futura ni uso ilimitado de una suscripción.

4.2.7 Worker descargable y adaptación al dispositivo

La configuración inicial emplea Qwen2.5-Coder-1.5B-Instruct Q4_K_M [23] mediante llama.cpp [28] y BGE-M3 en CPU. El catálogo de `setup-worker.py` permite descargar modelos con revisión y SHA-256 fijos, y la paleta permite seleccionar cualquier GGUF compatible. El razonamiento del vectorizador va desactivado: su tarea es describir, no conversar. El techo recomendado para una GPU dedicada es Qwen3 8B [30] a 4 bits; por encima solo tiene sentido en equipos con memoria unificada. Las recomendaciones de memoria del catálogo son orientativas y no equivalen a benchmarks de todos sus modelos.

Un dispositivo con más recursos puede evaluar un generador de mayor capacidad manteniendo el mismo recorrido. Una ficha más precisa puede aportar mejores datos para recuperar contexto, pero hay que medir calidad, latencia y memoria. La identificación del generador utiliza su nombre de archivo; sustituir pesos con el mismo nombre requiere invalidar la caché manualmente.

4.2.8 Selección de modelos preentrenados y delimitación del entrenamiento propio

Para el desarrollo se dispuso de un portátil con una RTX 3060 de 6 GB y de dos estaciones de trabajo adicionales accesibles por SSH con dos NVIDIA RTX 3090. Esa disponibilidad permite plantear inferencia y ajustes acotados de modelos existentes, pero no se realizó en esta versión un entrenamiento en esas estaciones.

Entrenar desde cero y ajustar un modelo preentrenado son tareas diferentes. Como referencia de escala, el informe de Qwen2.5-Coder describe un entrenamiento continuado sobre más de 5,5 billones de tokens [23]; esa cifra no es un requisito mínimo universal. Un ajuste mediante técnicas como QLoRA reduce las exigencias de memoria [24]. La presencia de dos GPU no constituye automáticamente un único espacio de memoria; el reparto del modelo y la comunicación requieren configuración específica.

El planteamiento inicial contemplaba destilar una red propia. Se decidió excluir ese entrenamiento del alcance final para concentrar el trabajo en la indexación, la recuperación y

su validación. Pesaron la preparación de un corpus propio revisado, el coste de iteración y el plazo disponible. No se dispone de una comparación que demuestre que un modelo propio rendiría necesariamente peor, ni de un presupuesto medido del entrenamiento descartado.

Quirón conserva la función prevista del worker mediante modelos preentrenados sustituibles: describir archivos y lógicas, producir datos para la vectorización y mantener un mapa consultable. El código propio integra y controla ese recorrido. La selección por dispositivo permite evolucionar el componente aprendido sin atribuir a este trabajo el entrenamiento de los pesos descargados. Un ajuste futuro quedaría condicionado a demostrar una mejora frente a esta base.

4.2.9 Metodología y reproducibilidad

Se distinguen cuatro niveles de afirmación: diseño, implementación inspeccionada, prueba funcional registrada y resultado experimental comparativo. Las salidas JSON y las capturas sirven como evidencia de recorridos concretos. Una prueba que pasa no acredita comportamientos que no ejerce. Cada evaluación registra revisión del código, modelos, corpus, preguntas, métricas y limitaciones, y los errores se conservan junto con los aciertos.

4.3 Recursos

Recurso	Uso y alcance de la evidencia
Portátil con RTX 3060 de 6 GB y 15 GiB de RAM utilizable	Equipo de desarrollo y de todas las pruebas locales registradas
Dos estaciones adicionales con dos RTX 3090	Disponibles; no se atribuyen resultados de entrenamiento
Rust, Sled, Qdrant, Neo4j y servicios de usuario de systemd	Editor, almacenamiento y procesos locales
Qwen2.5-Coder, llama.cpp y BGE-M3 sobre ONNX Runtime	Inferencia de fichas y embeddings
CLI de Claude y Codex, endpoints configurables	Acceso a asistentes; condiciones dependientes del proveedor
Python, Docker y Xvfb	Instalador, pruebas de entrega y pruebas de ventana sin pantalla

Tabla 5. Recursos empleados

El coste inicial incluye descargas: aproximadamente 1,15 GB para el runtime y el modelo inicial del worker, además de BGE-M3 y las imágenes de los almacenes. No se descargó un corpus para entrenar una red propia. La VRAM observada del servidor Qwen fue de unos 1 130 MiB en la prueba registrada; no representa su pico máximo ni el consumo de todas las configuraciones. Tras una incidencia de presión de memoria durante el desarrollo, las unidades de systemd limitan la memoria del cerebro y del worker, y el servidor de Qwen arranca con la caché de prompts acotada.

4.4 Presupuesto

Tipo de coste	Valor	Comentarios
Horas de trabajo en el proyecto	[COMPLETAR — horas totales]	Trabajo del autor; no han participado otras personas
Equipo técnico: portátil con RTX 3060 de 6 GB y 15 GiB de RAM	[COMPLETAR — valor de mercado] €	Equipo propio, no adquirido para el proyecto; en él se hicieron todas las pruebas registradas
Equipo técnico: dos estaciones con dos RTX 3090	[COMPLETAR — valor de mercado] €	Equipos propios accesibles por SSH; no se usaron para entrenar
Software utilizado	0 €	Rust, Sled, Qdrant, Neo4j, llama.cpp, ONNX Runtime, Python, Docker y los pesos de Qwen y BGE-M3 son de código abierto o de descarga gratuita bajo sus licencias
Suscripciones a asistentes (Claude y ChatGPT/Codex)	[COMPLETAR — gasto real por periodo] €	Necesarias para las rondas con proveedores reales; los servidores compatibles y Ollama no requieren suscripción
Estudios e informes	0 €	Todas las fuentes consultadas son de acceso abierto (arXiv y documentación pública)
Materiales empleados	0 €	Sin material de laboratorio; descargas de unos 1,15 GB para el runtime y el modelo del worker, además de BGE-M3 y las imágenes de los almacenes
Electricidad y almacenamiento	[COMPLETAR — estimación, si se incluye]	Coste de la inferencia local; no se ha medido el consumo

Tabla 6. Presupuesto del proyecto

No se presenta un coste total hasta completar las filas pendientes. La disponibilidad de software y pesos descargables no elimina las condiciones de licencia ni los costes operativos. La revisión de licencias de distribución acompaña al paquete.

4.5 Viabilidad y despliegue

Existe un paquete de instalación por usuario para Linux x86_64 que prepara configuración privada, servicios y lanzador. Requiere Python, Docker, systemd de usuario y acceso a las descargas. No es un paquete offline ni una distribución certificada para cualquier Linux. La comprobación mediante `--destdir` verifica archivos y configuración sin activar servicios; no sustituye una prueba de arranque en un equipo limpio.

El primer candidato, compilado en el equipo de desarrollo, exigía GLIBC_2.43 y no cargaba en una base Debian 13 con glibc 2.41. La compilación de entrega pasa a Ubuntu 24.04 (glibc 2.39), con imagen base fijada por digest y Rust 1.93.0, y el empaquetador rechaza símbolos superiores a esa versión. Ubuntu 22.04 se descartó porque el archivo estático de ONNX Runtime utiliza funciones de C23 que su glibc no proporciona. El paquete incluye cinco binarios (editor, cerebro, pasarela, indexador y herramienta del registro), las fuentes, la memoria y las evidencias. Los cinco binarios resuelven sus bibliotecas en Ubuntu 24.04 y Debian 13; en Ubuntu se abrió una ventana Xvfb a 1280 × 720 y se comprobó el instalador en staging. Esa prueba descubrió bibliotecas X11 cargadas dinámicamente que `ldd` no detecta (`libXcursor`, `libXi`); se añadieron al control previo del instalador. No se certifica el ciclo completo con Docker y systemd en una máquina virtual limpia. El paquete comprobado se compiló a partir de la revisión del 10 de septiembre; el script de empaquetado permite regenerarlo desde la revisión entregada.

El cerebro mantiene los almacenes y el worker: Qdrant y Neo4j se levantan justo antes del cerebro y se paran con él, sin arrancar al encender la máquina. Cerrar la ventana permite continuar el trabajo en segundo plano, y el apagado por inactividad tiene en cuenta las peticiones y los barridos activos.

4.6 Resultados y evaluación

4.6.1 Pruebas automatizadas

Los tres crates y el instalador disponen de pruebas automatizadas. La tabla recoge la ejecución final sobre la revisión entregada, después de retirar del código los adaptadores y el servicio de embeddings externo que ya no se usaban.

Componente	Pruebas aprobadas	Observaciones
Editor (workspace de siete crates)	329	Tres pruebas ignoradas por defecto; las que usan un servidor simulado necesitan acceso a localhost
Cerebro (--features full)	183	Incluye mutaciones del hash v2, rechazo atómico de lotes, cuatro escritores concurrentes, corrupción, migración y recuperación tras terminar un proceso
Pasarela	12	Selección explícita de proveedor por conversación y adaptadores vigentes
Entrega (instalador)	9	Docker simulado; no acreditan una instalación limpia con bases reales

Tabla 7. Pruebas automatizadas de la revisión entregada

Las pruebas del cerebro no son una prueba de corte eléctrico ni de todos los fallos de disco posibles. Un error de entrada/salida después de confirmar deja un resultado incierto; un reintento con el mismo identificador se rechaza si ya estaba guardado.

4.6.2 Integración del worker y del editor

Evidencia	Qué acredita	Límite
2026-09-05/w orker-integr acion.json	Dos proyectos sintéticos: consulta, cambios, borrados, exclusión de secretos y enlaces, caché	No mide precisión en proyectos arbitrarios
2026-09-05/a gentes/resum en.json	Rondas de interfaz con Claude, Codex, servidor local y escenarios compatibles simulados	Distinguir inferencia real de transporte simulado
2026-09-05/c onsulta-codi go.json	Consultas con fuentes, control de vigencia y uso del chat	Muestra pequeña; incluye respuestas incompletas
2026-09-10/w orker-integr acion.json	Repetición sobre dos proyectos temporales: autenticación, consultas, cambios, borrados, proyecciones y ausencia de inferencia sin cambios	Equipo de desarrollo
2026-09-10/g ui/gui-edit. json	Abrir, editar, guardar, deshacer y rehacer con estados intermedios, cerrar y reabrir conservando contenido e identidad, actualizar el índice	Una ventana, un archivo
2026-09-10-e stabilidad/w orker-integr acion.json	13 comprobaciones con Qwen, BGE-M3, Qdrant y Neo4j reales, incluidos filtros de vecinos entre proyectos y modelos	Equipo de desarrollo

Tabla 8. Evidencias funcionales registradas

La verificación histórica del registro de eventos contó 5 057 eventos y una cadena válida bajo el algoritmo inicial; el significado de esa validación queda limitado por el apartado 4.2.1. La herramienta `ledger_admin` se ensayó sobre una copia antes de aplicarla al servicio: conservó byte a byte los 5 284 eventos existentes y añadió un evento de anclaje, con 5 285 eventos válidos en la comprobación posterior. Una ejecución intermedia de la integración fue interrumpida por el apagado por inactividad del servicio, que detuvo sus almacenes; se conservó ese fallo y se repitió la prueba manteniendo el servicio activo.

4.6.3 Evaluación exploratoria de localización

El protocolo utiliza preguntas de localización con símbolos esperados identificados en el código antes de consultar el índice. Mide si el objetivo aparece entre los cinco primeros resultados vectoriales y en el conjunto ampliado por llamadas, y registra el rango de la primera coincidencia. Una coincidencia de símbolo es un indicador de localización; no demuestra que la respuesta explique correctamente su comportamiento. Se compara además el volumen de las fichas recuperadas con el texto de los archivos de código bajo el mismo tokenizador: es una medida del material de contexto, no del coste de una tarea completa ni de la facturación de un proveedor.

La ejecución se conserva en `docs/evidencias/2026-09-10/localizacion.json`, junto con el protocolo `docs/evaluacion/localizacion-v1.json` y el script `scripts/evaluate-index.py`. El conjunto de referencia contiene 140 archivos Rust versionados bajo `src/`, de hasta 512 KiB cada uno. La búsqueda se hizo sobre el índice del proyecto completo, con los modelos ya cargados.

Medida	Resultado observado
Preguntas con objetivo entre los cinco resultados directos (Hit@5)	10 de 10
Media del inverso del rango del objetivo, hasta cinco (MRR@5)	0,75
Preguntas con objetivo tras la expansión por llamadas	10 de 10; no aumenta los aciertos en esta muestra
Mediana de duración de la petición de búsqueda	0,111 s
Fichas devueltas, contando apariciones entre preguntas	75; todas coinciden con el proyecto solicitado y con el hash del archivo actual
Texto de referencia de los 140 archivos	420 881 tokens
Mediana del conjunto de fichas por pregunta, incluida expansión	2 491,5 tokens; 0,592 % del volumen de referencia

Tabla 9. Resultados de la evaluación de localización sobre diez preguntas

Hit@5 cuenta las preguntas cuyo símbolo esperado aparece en los cinco primeros resultados; MRR@5 premia que aparezca antes. Se empleó el tokenizador de Qwen2.5-Coder-1.5B-Instruct mediante `/tokenize`, sin tokens especiales. El informe conserva las preguntas, los objetivos, los resultados completos, los hashes del corpus y los tiempos, y no se detectaron cambios de esos archivos durante la evaluación.

El volumen menor de las fichas no se presenta como un porcentaje de ahorro económico: enviar todo el código es solo una referencia de tamaño. Las preguntas proceden del desarrollo, no de un conjunto independiente; tampoco se puntuó la fidelidad de cada resumen ni se comparó el éxito de una tarea completa. Los diez aciertos permiten repetir una prueba de localización, pero no estimar una precisión general ni atribuir una mejora al grafo.

4.6.4 Fidelidad de las fichas

Una inspección de las fichas de esos objetivos encontró errores concretos en la versión v4 del worker: `collect_calls` se describía como análisis de TypeScript aunque su integración utiliza Rust; `read_file` mencionaba 1 000 caracteres cuando la constante es 24 000; y `stable_id` situaba al final dos bytes cero que el código intercala como separadores. También había descripciones que terminaban a mitad de frase. Los ejemplos y los fragmentos de contraste se conservan en `fidelidad-observaciones.json`. Esta inspección forma parte del desarrollo, no de una valoración independiente ni de una tasa de error representativa, y confirma que la firma y el código deben prevalecer sobre el texto generado.

La versión v5 de las fichas, descrita en el apartado 4.2.3, se probó sobre una muestra de ocho funciones que comprendía los tres ejemplos problemáticos y cinco funciones sintéticas adicionales. Siete descripciones fueron aceptadas y una se sustituyó por una ficha estructural al no terminar una frase completa. Las citas y los hashes coincidieron con las fuentes, y los tres errores anteriores no reaparecieron. Las descripciones todavía pueden omitir ramas, unidades o efectos. No se establece una tasa de fidelidad semántica ni se extrapola el Hit@5 de v4 a v5 sin repetir la evaluación.

4.6.5 Proveedores desde la interfaz

Las dos CLI se comprobaron desde la ventana del editor sobre la misma función sintética de cuatro líneas, que duplica un entero mediante `checked_mul`. El worker produjo dos fichas, de archivo y de función, sin recurrir al analizador estructural. En ambos casos el chat recuperó las fichas con sus referencias y hash, el asistente solicitó leer el archivo mediante la herramienta del editor y devolvió `Some(14)` para la entrada siete y `None` para el máximo de u32. La CLI de Claude resolvió el alias `sonnet` como `claude-sonnet-5`; Codex utilizó `gpt-6-astra` con esfuerzo `xhigh`. El editor registró 12,8 y 32,8 segundos respectivamente; dos rondas no permiten comparar rendimiento general. Al pasar de Claude a Codex no cambiaron los procesos del cerebro ni del worker ni el archivo privado de configuración. Las ventanas cerraron normalmente y el código no se modificó. Las evidencias están en `docs/evidencias/2026-09-10-proveedores/` y `docs/evidencias/2026-09-10-codex/`.

El catálogo consultado con Codex CLI 0.153.0 anunciaba GPT-6 Astra, GPT-5.6 Sol, Terra y Luna, GPT-5.5 y GPT-5.3 Codex Spark. Son los modelos visibles en esa cuenta y fecha; no se presenta esa lista como disponibilidad universal. Una prueba de transporte frente a un servidor de respuestas simulado verificó el modelo, el esfuerzo, el contrato de lectura, la retirada de una credencial sintética del entorno y la ausencia de herramientas nativas con acceso al proyecto.

4.6.6 Validación pendiente

Quedan pendientes una evaluación independiente de la fidelidad de las fichas, tareas completas con y sin índice, una referencia de recuperación léxica o sintáctica, la repetición con varios repositorios y una instalación limpia completa en otro equipo. El benchmark `ledger_bench.rs` mide lectura y escritura del registro; no mide ahorro de contexto, detección de duplicados ni durabilidad de cada evento frente a pérdida de alimentación.

Capítulo 5. DISCUSIÓN

La elección de modelos preentrenados permite aprovechar capacidades desarrolladas por terceros y concentrar el trabajo propio en el recorrido del producto. La separación entre generador y embeddings permite estudiar mejoras de cada componente por separado. Las dos estaciones adicionales amplían las posibilidades de experimentación; su existencia no convierte en necesario entrenar un modelo para esta entrega.

Las fichas ya permiten describir archivos y determinadas lógicas. Las responsabilidades agregadas por carpeta y la detección automática de duplicados no están implementadas. La búsqueda por similitud puede proporcionar candidatos para una revisión, pero similitud no equivale a identidad de comportamiento. El hash comprueba la correspondencia con el archivo; no valida la semántica del resumen.

Las garantías deben expresarse con precisión. El índice depende de los archivos y de la caché local; el registro de eventos no contiene todo lo necesario para reconstruirlo. Las relaciones de llamadas emplean aproximaciones; la recuperación ampliada aplica filtros explícitos y conserva metadatos, pero eso no convierte las aristas en un análisis semántico completo. Los resultados de aislamiento sobre dos proyectos son evidencia acotada, no una prueba formal de todas las consultas posibles.

La validación comparativa puede comenzar con un presupuesto pequeño. Contar tokens offline no requiere pagar una petición que incluya todo el repositorio, pero por sí solo tampoco demuestra calidad equivalente. Una evaluación de tareas debe contabilizar instrucciones, fichas, lecturas adicionales, historial, resultados de herramientas y respuesta. No se presenta el coste de una prueba que no se realizó como evidencia de ahorro.

Por último, la reproducibilidad de la entrega exige que Markdown, DOCX y PDF, fuentes, binarios y evidencias indiquen su revisión y su alcance. Las capturas muestran una ejecución; el paquete se verifica por separado con su inventario de hashes.

Capítulo 6. CONCLUSIONES

6.1 Conclusiones del trabajo

Se ha construido un prototipo de editor nativo que integra un worker local, fichas de archivos y de lógica Rust, búsqueda vectorial, un grafo parcial y referencias comprobables contra el código vigente. El sistema incorpora la descarga y selección de modelos compatibles y distintas conexiones con asistentes. Las pruebas registradas demuestran varios recorridos completos de integración y han permitido identificar y corregir fallos de calidad de las fichas, de integridad del registro y de recursos del equipo.

El entrenamiento propio fue un objetivo inicial reformulado de manera explícita. La contribución implementada se encuentra en la integración, el mantenimiento incremental y el control del contexto que recibe el asistente. El estudio matemático del Anexo A conserva valor como exploración de alternativas, sin presentarse como una nueva arquitectura entrenada o evaluada.

La hipótesis de reducción de contexto manteniendo la calidad de tareas completas sigue abierta. Siguen pendientes la reconstrucción completa del índice desde el registro y un análisis completo de dependencias. El registro v2 cubre todos los campos y confirma escrituras transaccionales; su anclaje histórico y la prueba de interrupción tienen los límites explícitos del apartado 4.2.1. El grado de cumplimiento se recoge en el Capítulo 3 para que los resultados puedan evaluarse sin confundir el prototipo con la totalidad del diseño inicial.

6.2 Conclusiones personales

[COMPLETAR — reflexión personal del autor sobre el aprendizaje, las decisiones tomadas y las dificultades encontradas durante el proyecto.]

Capítulo 7. FUTURAS LÍNEAS DE TRABAJO

- Ampliar las pruebas del registro v2 a fallos de almacenamiento y pérdida de alimentación, y estudiar puntos de control externos si se requiere detectar la reescritura de toda la base.
- Registrar cambios de código suficientes para reconstruir las proyecciones desde evidencia histórica y probar su equivalencia lógica.
- Resolver mejor módulos, tipos y llamadas, y ampliar las pruebas de recuperación y aislamiento a más repositorios.
- Incorporar reranking y presupuesto de contexto con el tokenizador del asistente, medidos en tareas completas.
- Comparar generadores y cuantizaciones por fidelidad, latencia y memoria antes de seleccionar una configuración superior.
- Evaluar LoRA/QLoRA o destilación especializada sobre ejemplos revisados, si aporta mejoras frente al modelo base.
- Detectar candidatos a duplicados mediante estructura y similitud, con confirmación humana y métricas de falsos positivos.
- Presentar responsabilidades agregadas por carpetas y ampliar la extracción sintáctica a otros lenguajes.
- Certificar instalaciones limpias, actualizar paquetes y completar la revisión de licencias y portabilidad.

Capítulo 8. REFERENCIAS

Los informes técnicos de modelos se citan como resultados de sus autores; no son mediciones reproducidas en este trabajo.

[1] DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model", arXiv:2405.04434, 2024.

[2] DeepSeek-AI, "DeepSeek-V3 Technical Report", arXiv:2412.19437, 2024.

[3] DeepSeek-AI, "Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention", arXiv:2502.11089, 2025.

[4] MiniMax, "MiniMax-01: Scaling Foundation Models with Lightning Attention", arXiv:2501.08313, 2025.

[5] MiniMax, "MiniMax-M1", arXiv:2506.13585, 2025.

[6] Ling Team, "Every Attention Matters: An Efficient Hybrid Architecture for Long-Context Reasoning", arXiv:2510.19338, 2025. <https://arxiv.org/abs/2510.19338>

[7] Y. Pan, Y. An, Z. Li, Y. Chou, R. Zhu, X. Wang, M. Wang, J. Wang y G. Li, "Scaling Linear Attention with Sparse State Expansion", arXiv:2507.16577, 2025. <https://arxiv.org/abs/2507.16577>

[8] DeepSeek-AI, "DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models", arXiv:2401.06066, 2024.

[9] L. Wang, H. Gao, C. Zhao, X. Sun y D. Dai, "Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts", arXiv:2408.15664, 2024.

[10] G. Hinton, O. Vinyals, J. Dean, "Distilling the Knowledge in a Neural Network", arXiv:1503.02531, 2015.

[11] Y. Kim, A. M. Rush, "Sequence-Level Knowledge Distillation", arXiv:1606.07947, 2016.

[12a] Y. Gu et al., "MiniLLM: Knowledge Distillation of Large Language Models", arXiv:2306.08543, 2023.

[12b] R. Agarwal et al., "On-Policy Distillation of Language Models (GKD)", arXiv:2306.13649, 2023.

[12c] T. Wu, C. Tao, J. Wang, R. Yang, Z. Zhao y N. Wong, "Rethinking Kullback-Leibler Divergence in Knowledge Distillation for Large Language Models", arXiv:2404.02657, 2024. <https://arxiv.org/abs/2404.02657>

[13] A. van den Oord, Y. Li, O. Vinyals, "Representation Learning with Contrastive Predictive Coding", arXiv:1807.03748, 2018.

- [14] L. Wang et al., "Improving Text Embeddings with Large Language Models (e5-mistral)", arXiv:2401.00368, 2024.
- [15] D. Kryvosheieva, S. Sturua, M. Günther y H. Xiao, "Efficient Code Embeddings from Code Generation Models", arXiv:2508.21290, 2025. <https://arxiv.org/abs/2508.21290>
- [16] A. Kusupati et al., "Matryoshka Representation Learning", arXiv:2205.13147, 2022.
- [17] Qdrant, "Qdrant — Vector Database", documentación. <https://qdrant.tech> (consultado en septiembre de 2026).
- [18] Neo4j, "Neo4j Graph Database", documentación. <https://neo4j.com> (consultado en septiembre de 2026).
- [19] BAAI, modelos bge-m3 y bge-reranker-v2-m3. <https://huggingface.co/BAAI> (consultado en septiembre de 2026).
- [20] J. Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration", arXiv:2306.00978, 2023.
- [21] E. Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers", arXiv:2210.17323, 2022.
- [22] Microsoft, "ONNX Runtime", documentación. <https://onnxruntime.ai> (consultado en septiembre de 2026).
- [23] B. Hui et al., "Qwen2.5-Coder Technical Report", arXiv:2409.12186, 2024. <https://arxiv.org/abs/2409.12186>
- [24] T. Dettmers, A. Pagnoni, A. Holtzman y L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs", arXiv:2305.14314, 2023. <https://arxiv.org/abs/2305.14314>
- [25] F. Zhang et al., "RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation", arXiv:2303.12570, 2023. <https://arxiv.org/abs/2303.12570>
- [26] W. Liu et al., "GraphCoder: Enhancing Repository-Level Code Completion via Code Context Graph-based Retrieval and Language Model", arXiv:2406.07003, 2024. <https://arxiv.org/abs/2406.07003>
- [27] S. Ouyang et al., "RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph", arXiv:2410.14684, 2024. <https://arxiv.org/abs/2410.14684>
- [28] ggml-org, "llama.cpp server", documentación y código. <https://github.com/ggml-org/llama.cpp/tree/master/tools/server> (consultado en septiembre de 2026).
- [29] J. Chen et al., "M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation", arXiv:2402.03216, 2024. <https://arxiv.org/abs/2402.03216>

[30] Qwen Team, "Qwen3 Technical Report", arXiv:2505.09388, 2025.
<https://arxiv.org/abs/2505.09388>

[31] M. Brunsfeld et al., "Tree-sitter: an incremental parsing system for programming tools", documentación y código. <https://tree-sitter.github.io> (consultado en septiembre de 2026).

[32] T. Kemp et al., "sled: an embedded database", código. <https://github.com/spacejam/sled> (consultado en septiembre de 2026).

Capítulo 9. ANEXOS Y TRAZABILIDAD

- **Anexo A — Estudio de alternativas para la red obrera:** se incorpora a continuación de este capítulo desde `docs/ESTUDIO_RED_OBRERA.md`. Se conserva como propuesta inicial, con una nota que aclara su relación con la implementación.
- **Anexo B — Informes técnicos de cierre:** `docs/CIERRE_TFM_2026-09-10.md` (contraste entre código y memoria, con los límites técnicos encontrados), `docs/ESTABILIZACION_2026-09-10.md` (correcciones del registro, de las fichas y del paquete) y `docs/CODEX_EN_QUIRON_2026-09-10.md` (integración de Codex).
- **Anexo C — Manual e instalación:** `docs/MANUAL.md` (manual de uso, accesible desde la aplicación), `docs/GUIA_EVALUACION.md` (guía para evaluar la aplicación en otro equipo) y `deploy/LINUX.md` (requisitos y límites del paquete Linux). El código es la referencia para resolver cualquier discrepancia con la documentación.
- **Anexo D — Evidencias:** `docs/evidencias/2026-09-05/` para las pruebas de la primera integración y `docs/evidencias/2026-09-10*/` para las del cierre. Los archivos de cada ejecución indican su alcance; una captura o una simulación no se presenta como ensayo de instalación limpia.
- **Anexo E — Fuentes y artefactos:** repositorio de entrega [COMPLETAR — URL], revisión etiquetada `entrega-tfm`. Este Markdown es la fuente de las exportaciones DOCX y PDF, generadas con `scripts/export-memory.py`; el paquete binario se construye con `scripts/package-linux-portable.sh` y lleva su inventario de hashes.

Anexo A. Estudio de alternativas para la red obrera

Estudio: la matemática de la red obrera de Quirón

Algoritmos y ecuaciones de los LLM recientes, y cómo reutilizarlos en nuestro worker
TFM Quirón · Anexo A de la memoria · 10 de julio de 2026

Nota sobre el alcance. Este estudio recoge las alternativas que analicé para el diseño inicial de la red obrera. La versión entregada de Quirón usa modelos preentrenados descargables (Qwen como generador de fichas y BGE-M3 para los embeddings); el backbone que aquí se propone no llegó a entrenarse ni a exportarse a ONNX. Las etiquetas de «verificación adversarial» describen el procedimiento que seguí en julio al contrastar las fuentes; no sustituyen una reproducción experimental. El índice de código de la versión final se mantiene desde los archivos y una caché local, no como reconstrucción completa desde el registro de eventos. Los resultados vigentes están en la memoria.

Este documento extrae **la matemática** de los mecanismos con los que se han construido los LLM recientes —no compara modelos— y explica, para cada uno: **(a)** la ecuación esencial, **(b)** por qué funciona, **(c)** la referencia de arXiv, y **(d)** cómo se aplica a nuestro worker. El objetivo es reutilizar esas matemáticas en **una sola red pequeña, local, destilable y exportable a ONNX/Rust**.

Cómo leer las etiquetas de evidencia

Fue un barrido de fuentes primarias con verificación adversarial (tres pasadas, ~75 afirmaciones sobre >40 fuentes). Cada mecanismo lleva una etiqueta:

- **Verificado** — confirmado 3-0 en la verificación adversarial de este barrido.
- **Fundamental** — matemática canónica establecida (referencia original); **no re-verificada** adversarialmente aquí, pero es estándar de libro.
- **Principio de diseño** — la idea es reutilizable, pero **no la desplegamos tal cual** por fricción con ONNX/Rust o por tamaño.
- **Cuestión abierta** — debate activo en la literatura; se declara como tal.

Honestidad metodológica: las cifras de los *technical reports* (−93,3 % de KV-cache, 25 % de FLOPs, «degradación negligible») son **autoreportadas** por sus autores. Las de complejidad ($O(n \cdot d^2)$, tamaño de estado $d \times d$) son propiedades deterministas de la arquitectura. Se distingue donde importa.

0. Marco del diseño inicial

El diseño inicial proponía un daemon asociado al proyecto, con dos proyecciones —vectores en Qdrant y grafo en Neo4j— reconstruibles desde un registro de cambios. Una sola red pequeña habría aportado descripciones y representaciones. Ese diseño no se realizó íntegramente: la versión de cierre usa archivos y caché local, Qwen para describir y BGE-M3 para vectorizar. Las limitaciones de la cadena de hashes se explican en §4.2.1 de la memoria. El worker no ejecuta herramientas, pero una descripción incorrecta sí puede inducir errores en el asistente que la recibe; reconstruir el grafo no valida esa descripción.

Sus tres prioridades ordenan a qué sirve cada matemática de abajo: **(1) aislar por proyecto, (2) enlazar/recuperar vectores a escala, (3) etiquetar qué hace cada archivo sin alucinar.**

FAMILIA 1 — Atención eficiente y gestión del KV-cache

El problema: la atención estándar cuesta $O(n^2 \cdot d)$ y su KV-cache crece **lineal** con la longitud. Tres formas de romperlo.

1.1 Multi-head Latent Attention (MLA) — comprimir el KV · Verificado

(a) Ecuación. En vez de cachear K y V por cabeza, se comprimen juntos a un latente de bajo rango y se reconstruyen al vuelo:

$$\begin{array}{ll} c_t^{KV} = W^{DKV} \cdot h_t & \text{con } d_c \ll d_h \cdot n_h \quad (\text{down-projection}) \\ k_t^C = W^{UK} \cdot c_t^{KV} & v_t^C = W^{UV} \cdot c_t^{KV} \quad (\text{up-projections}) \\ k_t^R = \text{RoPE}(W^{KR} \cdot h_t) & (\text{clave posicional RoPE, desacoplada y compartida}) \end{array}$$

Solo se **cachea** c_t^{KV} (**pequeño**) y la **clave RoPE** k_t^R , no K/V completos.

(b) Por qué funciona. K y V de un token viven en un subespacio de dimensión mucho menor que $d_h \cdot n_h$; una proyección de bajo rango los captura casi sin pérdida. El RoPE se saca aparte porque su rotación **no puede absorberse** en las up-projections (depende de la posición). Resultado: **-93,3 %** de KV-cache respecto a DeepSeek-67B con calidad comparable a MHA.

(c) DeepSeek-V2 arXiv:2405.04434; DeepSeek-V3 arXiv:2412.19437.

(d) Aplicación al worker. Son solo *matmuls* → **exportable a ONNX**. Un KV-cache pequeño abarata el daemon en inferencia autoregresiva en Rust. El truco de absorción de matrices reduce memoria. (*Matiz : la variante con absorción y RoPE desacoplado no es trivial de exportar; el principio low-rank sí.*)

1.2 Native Sparse Attention (NSA) — atención dispersa entrenable · Verificado

(a) Ecuación. Tres ramas combinadas por un *gate* aprendido:

$$o_t^* = \sum_{c \in \{cmp, slc, win\}} g_t^{\{c\}} \cdot \text{Attn}(q_t, K_t^{\{c\}}, \tilde{V}_t^{\{c\}})$$

$$g_t^{\{c\}} = \text{sigmoid}(\text{MLP}(x_t)) \in [0,1]$$

- **cmp**: compresión gruesa de bloques de tokens (contexto global barato).
- **slc**: selección fina de los **top-n bloques** más relevantes.
- **win**: ventana deslizante (contexto local reciente).

Truco clave: los **scores** de selección salen **gratis** de la rama de compresión, $p_t^{\{cmp\}} = \text{softmax}(q_t^T K_t^{\{cmp\}})$, y se reutilizan para rankear los bloques.

(b) Por qué funciona. Es **entrenable end-to-end** (el *gate* es diferenciable), no una poda solo-inferencia; y es *hardware-aligned* (carga bloques KV contiguos para GQA, aprovechando Tensor Cores).

(c) arXiv:2502.11089 (DeepSeek, ACL 2025 best paper).

(d) Aplicación al worker. El **gating diferenciable de ramas** es reutilizable: permite atender selectivamente a los vectores de archivos relacionados dentro de un proyecto **sin coste cuadrático**. La selección top-n sirve a la prioridad 2.

1.3 Atención lineal / lightning + SSE — estado de tamaño fijo · Verificado

(a) Ecuación. El «truco del producto por la derecha» reordena la asociatividad:

$$O = \text{Norm}((Q K^T) V) \rightarrow O = \text{Norm}(Q (K^T V)) \quad O(n^2 \cdot d) \rightarrow O(n \cdot d^2)$$

La asociatividad anterior se refiere al producto sin una no linealidad entre QK^T y V : no permite mover la softmax de la atención convencional. Una formulación causal requiere además su máscara o recurrencia correspondiente.

En forma recurrente con decaimiento, el estado es una matriz **d×d de tamaño fijo**:

$$kv_t = \lambda \cdot kv_{t-1} + k_t^T v_t \quad o_t = q_t \cdot kv_t$$

(b) Por qué funciona. $K^T V$ es un estado $d \times d$ que **no crece con la longitud**: coste **constante por paso** en inferencia. Lo que se pierde es *retrieval* asociativo (comprimir a estado fijo difumina). Por eso se **hibrida**: 1 bloque de atención completa cada 7 lineales (7:1). **SSE** (arXiv:2507.16577) mitiga la pérdida con una actualización de estado **top-k dura** (filas dispersas) que reduce la interferencia entre informaciones y **desacopla la capacidad de estado del nº de parámetros**.

(c) MiniMax-01 arXiv:2501.08313; MiniMax-M1 arXiv:2506.13585; Ring-linear arXiv:2510.19338; SSE arXiv:2507.16577.

(d) Aplicación al worker — la pieza más importante de esta familia. El estado recurrente $d \times d$ es **directamente reutilizable y exportable a ONNX**: da un **encoder de coste constante por token**, ideal para un daemon que procesa archivos **en streaming** sin re-materializar matrices $n \times n$. Para «enlazar a escala» (prioridad 2) conviene un backbone

mayormente lineal con pocas capas de atención completa. El top-k duro de SSE ayuda a la prioridad 3 (no mezclar lógicas de archivos distintos → menos alucinación). *(el kernel lightning I/O-aware con tiling no se reproduce fácil en Rust; se toma la forma recurrente, no el kernel.)*

FAMILIA 2 — Mezcla de expertos (MoE) y enrutado

2.1 DeepSeekMoE: experto compartido + expertos de grano fino · Verificado

(a) Ecuación.

```
h_t' = u_t + Σ_{i=1}^{N_s} FFN_i^{shared}(u_t) + Σ_{i=1}^{N_r} g_{i,t} · FFN_i^{routed}(u_t)
s_{i,t} = sigmoid(u_t^T e_i)                                (afinidad token-experto)
g_{i,t} = normaliza( s_{i,t} si i ∈ TopK_r, si no 0 )
```

DeepSeek-V3: $N_s=1$ compartido, $N_r=256$ enrutados, $K_r=8$ activos → **671 B totales, 37 B activados por token.**

(b) Por qué funciona. El experto **compartido** captura lo común; los de **grano fino** se especializan. Así se **desacopla la capacidad (parámetros) del cómputo por token.**

(c) DeepSeekMoE arXiv:2401.06066; DeepSeek-V2/V3 arXiv:2405.04434 / 2412.19437.

(d) Aplicación al worker. Principio de diseño, no despliegue. Los modelos MoE de la escala citada exceden el equipo disponible; no se concluye que todo MoE sea inviable. Su *routing* requiere estudiar la exportación a ONNX/Rust. El patrón «**una cabeza compartida + cabezas especializadas por tipo de lógica**» inspira la estructura de **nuestra única red** sin necesidad de *routing* disperso.

2.2 Balanceo de carga SIN pérdida auxiliar · Verificado

(a) Ecuación. Se añade un **sesgo por experto** b_i solo a la **decisión** de enrutado, nunca al peso de mezcla:

```
enruta i <=> (s_{i,t} + b_i) ∈ TopK                (el peso sigue siendo s_{i,t}, sin sesgo)
b_i ← b_i - γ si el experto i está sobrecargado
b_i ← b_i + γ si está infrautilizado                (γ = 0.001)
```

(b) Por qué funciona. La pérdida auxiliar clásica inyecta **gradientes de interferencia** que dañan el objetivo de lenguaje. Aquí el sesgo cambia **qué** experto se elige sin contaminar **cuánto** pesa, así el balanceo **no altera el objetivo**. Eleva el techo de rendimiento.

(c) Loss-Free Balancing arXiv:2408.15664; DeepSeek-V3 arXiv:2412.19437.

(d) Aplicación al worker. Si la red usa varias cabezas, este **control por sesgo** es un *feedback loop* barato para repartir carga **sin término de pérdida extra** — más simple de implementar y de exportar.

FAMILIA 3 — Destilación profesor → estudiante

Hilo conductor: **nuestro profesor se accede por login (Codex/Claude), sin API de pago y sin logits**. La pregunta matemática es: ¿qué destilación es posible si solo tenemos el **texto** que devuelve el profesor?

3.1 KD clásica (Hinton): *soft targets* con temperatura · Fundamental

(a) Ecuación.

$$L = (1 - \alpha) \cdot \text{CE}(y, \sigma(z_s)) + \alpha \cdot T^2 \cdot \text{KL}(\sigma(z_t / T) \parallel \sigma(z_s / T))$$

(b) **Por qué funciona.** La temperatura T **suaviza** el *softmax* del profesor y expone su «conocimiento oscuro» (qué clases considera parecidas). El factor T^2 compensa que los gradientes del término *soft* escalan como $1/T^2$, para que su magnitud iguale la del término *hard*.

(c) Hinton et al. arXiv:1503.02531. (no re-verificado en este barrido; fórmula canónica.)

(d) **Aplicación al worker. No aplicable directamente:** exige los **logits** del profesor z_t , que un profesor por login **no expone**.

3.2 Sequence-Level KD — la vía viable sin logits · Verificado

(a) **Ecuación.** El objetivo exacto suma sobre todas las secuencias (intratable); se aproxima por la **moda** del profesor, hallada con *beam search* $\hat{y}$:

$$L_{\{\text{SEQ-KD}\}} = - \sum_{\{t\}} q(t|s) \cdot \log p(t|s) \approx - \log p_s(t = \hat{y} | s)$$

Es decir: **entrenar al estudiante con cross-entropy sobre el TEXTO que genera el profesor.**

(b) **Por qué funciona.** Reduce la destilación a *maximum likelihood* sobre las respuestas del profesor. Procedimiento: (1) el profesor genera, (2) recoges su texto, (3) entrenas al estudiante con CE sobre ese corpus. **Solo necesita texto.** (Matiz honesto: la idea de que «la moda captura casi toda la masa» fue **refutada 0-3** en la verificación; el método se sostiene por **tratabilidad**, no porque esté probado que concentra la probabilidad.)

(c) Kim & Rush arXiv:1606.07947.

(d) **Aplicación al worker. Directamente reutilizable — es nuestra vía.** Es exactamente el subcaso *off-policy por respuestas* que un profesor por login sí permite. El estudiante puede ser ~10× más pequeño/rápido con poca pérdida.

3.3 Reverse-KL (MiniLLM) y GKD — por qué NO nos sirven · Verificado

- **MiniLLM** minimiza el **KL inverso** $KL(q_{\text{estudiante}} \parallel p_{\text{profesor}})$, *mode-seeking* (evita que el estudiante ponga masa donde el profesor casi no la tiene). Su gradiente es un **policy-gradient tipo REINFORCE**: $\nabla_{\theta} L = E_{\{x \sim q_s\}} [\nabla_{\theta} \log q_s(x) \cdot (\log q_s(x) - \log p_t(x))]$. Es **on-policy** y necesita **p_t token a token** → no viable solo con texto. arXiv:2306.08543.
- **GKD** generaliza con un objetivo mixto ponderado por λ ($\lambda=0$ off-policy sobre datos fijos; $\lambda=1$ on-policy sobre lo que genera el estudiante), pero **requiere los logits del profesor** sobre esas secuencias. no viable sin logits. El **único subcaso** que sobrevive sin logits es $\lambda=0$ con *targets* = secuencias del profesor... que **degenera en Sequence-Level KD**. arXiv:2306.13649.

3.4 Matiz: ¿reverse-KL es realmente mejor? · Cuestión abierta

La dicotomía *mode-seeking* (reverse) vs *mode-averaging* (forward) **presupone** distribuciones continuas y estudiante unimodal. En LLM las distribuciones son **discretas** (*softmax* sobre vocabulario) y q no es unimodal, así que la motivación de MiniLLM está **en disputa**: forward y reverse KL **comparten el mismo óptimo** $q_{\theta} = p$. Conclusión práctica: **no sobre-invertir en reverse-KL**; para un worker destilado solo con respuestas, **Sequence-Level KD (forward implícito por MLE) es lo defendible**. arXiv:2404.02657 (COLING 2025).

FAMILIA 4 — Cuantización y representación

4.1 AWQ — cuantizar protegiendo canales por activación · Verificado

(a) **Ecuación.** Reescalado equivalente por canal (sin *mixed precision*):

$$Wx \approx Q(W \cdot s) \cdot (x / s) \quad (s > 1 \text{ en los canales salientes})$$

$$s^* = \operatorname{argmin}_s \| Q(W \cdot s)(x / s) - Wx \|$$

(b) **Por qué funciona.** El error relativo de redondeo de un canal escala como $1/s$; subir s en los canales **salientes** reduce su error. Y los canales importantes se identifican por la **magnitud de la ACTIVACIÓN**, no del peso (seleccionar por peso rinde casi como al azar). Ej.: OPT-6.7B, proteger 1 % por activación baja la perplejidad 23.54 → 11.36. Es **PTQ**: no reentrena.

(c) AWQ arXiv:2306.00978 (MLSys 2024 best paper).

(d) **Aplicación al worker. Directamente reutilizable** para cuantizar a INT4 el backbone del worker y **caber y correr** en 48 GB / producción.

4.2 GPTQ — cuantización PTQ de segundo orden · Verificado

(a) **Ecuación.** Reconstrucción por capa con información del Hessiano (OBS):

$$\underset{\delta_F}{\operatorname{argmin}}_{\{\hat{W}\}} \|W X - \hat{W} X\|^2 \quad H_F = 2 \cdot X_F X_F^T$$

$$\delta_F = - (w_q - \operatorname{quant}(w_q)) / [H_F^{-1}]_{qq} \cdot (H_F^{-1})_{:,q} \quad (\text{redistribuye el error})$$

(b) **Por qué funciona.** La información de segundo orden **redistribuye el error de redondeo** a los pesos aún no cuantizados. Como en capas grandes el **orden importa poco**, se cuantizan todas las filas en el mismo orden $\rightarrow$ coste $O(d \cdot \text{col}^2)$ en vez de por peso. Cuantiza GPT-175B a 3-4 bits en ~ 4 GPU-h.

(c) GPTQ arXiv:2210.17323 (ICLR 2023).

(d) **Aplicación al worker.** Alternativa/complemento de AWQ para PTQ del backbone.

4.3 InfoNCE — la pérdida que da buenos embeddings · Fundamental

(a) **Ecuación.**

$$L = -\log \left[\exp(\operatorname{sim}(q, k^+)/\tau) / \sum_i \exp(\operatorname{sim}(q, k_i)/\tau) \right]$$

(b) **Por qué funciona.** Acerca el ancla q a su positivo k^+ y la aleja de los negativos k_i (los demás del *batch*). La temperatura τ controla cuánto se castiga a los negativos difíciles. Es aprendizaje contrastivo estándar.

(c) van den Oord et al. (CPC) arXiv:1807.03748. (*canónico, no re-verificado.*)

(d) **Aplicación al worker.** Base para entrenar los **embeddings de código 1024-dim** que pueblan Qdrant (prioridad 2). Positivos = pares código $\leftrightarrow$ resumen o código $\leftrightarrow$ código equivalente.

4.4 Last-token pooling — de decoder causal a embedding · Fundamental

(a) **Idea.** En un *decoder* causal, el **estado del último token** $e = h_L$ resume la secuencia, porque la atención causal deja que ese token «vea» todos los anteriores. Es cómo modelos como e5-mistral / gte-Qwen / jina-code-embeddings producen un vector desde un backbone autoregresivo.

(c) jina-code-embeddings arXiv:2508.21290; e5-mistral arXiv:2401.00368. (*no re-verificado.*)

(d) **Aplicación al worker.** Permite **reutilizar el mismo backbone destilado** tanto para etiquetar (generar) como para vectorizar (embedding), alineando ambas representaciones. Cuando tocamos un archivo, su vector = h_L de su resumen.

4.5 Matryoshka (MRL) — un vector, muchas dimensiones · Fundamental

(a) **Ecuación.** Se entrena con la suma de la pérdida sobre **prefijos anidados** del vector:

$$L = \sum_{m \in \{8, 16, 32, \dots, 1024\}} c_m \cdot L_{\text{task}}(z_{\{1:m\}})$$

(b) Por qué funciona. Al forzar que **cada prefijo** $z_{\{1:m\}}$ sea por sí solo un buen embedding, la información se ordena por importancia en las primeras dimensiones. Así el **mismo vector se puede truncar** a 256 o 64 dim **sin reentrenar**, perdiendo poca calidad.

(c) MRL arXiv:2205.13147. *(no re-verificado.)*

(d) Aplicación al worker. Vectores **1024-dim truncables**: búsqueda barata en dimensión baja + reranking en dimensión alta. Escala la prioridad 2 sin duplicar índices.

Síntesis — la red obrera propuesta

Combinando **solo lo reutilizable**, una **única red pequeña** con esta forma:

- **Backbone híbrido mayormente lineal** (estado recurrente $d \times d$, §1.3) con **pocas capas de atención completa** (ratio $\sim 7:1$). El componente recurrente tiene coste constante por paso para dimensiones fijas; el modelo completo conserva la dependencia de la longitud en sus capas de atención completa. Su exportación y rendimiento en ONNX requieren implementación y medición.
- **Una cabeza compartida + cabezas por tipo de lógica** (inspiración del experto compartido de §2.1, *sin routing* MoE real) para etiquetar.
- **Embeddings de código con InfoNCE (§4.3) + last-token pooling (§4.4) + Matryoshka a 1024-dim (§4.5)** → una representación que podría almacenarse en Qdrant. Compartir dimensión no la hace semánticamente compatible con bge-m3; requiere entrenamiento, evaluación y una colección/configuración coherente.
- **Entrenamiento por Sequence-Level KD (§3.2)** desde el profesor por login (solo texto): una vía de destilación viable sin logits, no ejecutada en esta versión.
- **Despliegue cuantizado con AWQ INT4 (§4.1)** para caber en 48 GB y correr en el mismo ort de Rust que ya usa el cerebro para BGE-M3.

Qué es reutilizable y qué es solo principio de diseño

Mecanismo	Veredicto para el worker
Estado lineal recurrente $d \times d$ (§1.3)	Directamente reutilizable (encoder ONNX)
Selección top-n / gating de NSA (§1.2)	Reutilizable (atender a vectores relacionados)
Sequence-Level KD (§3.2)	Nuestra vía de destilación
AWQ INT4 (§4.1) / GPTQ (§4.2)	Reutilizable (caber y correr)
InfoNCE + last-token + Matryoshka (§4.3–4.5)	Reutilizable (embeddings 1024-dim)
MLA / compresión de KV (§1.1)	Idea sí; export ONNX con fricción
MoE real / routing (§2.1)	Solo el patrón de cabezas; no desplegar
Kernel <i>lightning</i> I/O-aware (§1.3)	Forma recurrente sí; kernel no
Reverse-KL / GKD (§3.3)	Necesitan logits del profesor
KD clásica con temperatura (§3.1)	Necesita logits del profesor

Tabla 10. Qué es reutilizable y qué es solo principio de diseño

Cuestiones abiertas

- **Sin respaldo primario en este barrido** (fórmulas fundamentales, incluidas por canónicas pero **no re-verificadas** aquí): KD con temperatura T^2 (§3.1), InfoNCE (§4.3), last-token pooling (§4.4), Matryoshka (§4.5). Conviene citarlas como establecidas, no como «verificadas por nosotros».
- forward **vs** reverse **KL** es **debate activo** (§3.4): no cerrar la tesis de MiniLLM como hecho.
- **Fricción real ONNX/Rust** de pesos cuantizados por canal (AWQ INT4) y de MLA/MoE **no está medida** en fuentes primarias: hay que probarlo nosotros.
- **Fidelidad anti-alucinación** de resúmenes *code-to-text* pequeños: no hay evidencia primaria; se medirá con un conjunto etiquetado propio.

Referencias del anexo

Atención y KV: MLA — DeepSeek-V2 2405.04434, DeepSeek-V3 2412.19437 · NSA 2502.11089 · Lightning/lineal — MiniMax-01 2501.08313, MiniMax-M1 2506.13585, Ring-linear 2510.19338 · SSE 2507.16577. **MoE:** DeepSeekMoE 2401.06066 · Loss-Free Balancing 2408.15664. **Destilación:** Hinton 1503.02531 · Sequence-Level KD 1606.07947 · MiniLLM 2306.08543 · GKD 2306.13649 · Forward/Reverse KL 2404.02657. **Cuantización y representación:** AWQ 2306.00978 · GPTQ 2210.17323 · InfoNCE 1807.03748 · e5-mistral 2401.00368 · jina-code-embeddings 2508.21290 · Matryoshka 2205.13147.

Síntesis de la propuesta inicial

Se estudiaron **estado lineal recurrente**, **Sequence-Level KD**, **AWQ** e **InfoNCE + last-token pooling + Matryoshka** como componentes de una red futura. La combinación no fue entrenada ni validada en ONNX/Rust. La versión del producto utiliza generador y embeddings preentrenados separados. MLA, MoE y reverse-KL permanecen como alternativas del estudio con las limitaciones descritas sobre el profesor sin logits.